

Building Agentic AI Applications with OpenAI Agents SDK

→ orchestration - when multiple agents work together and their outputs are combined to single op.

1. Intro to OpenAI Agents SDK
2. Creating a single agent
3. Multi-agents systems (orchestration)
 - a. Hand-off Pattern (~~Decentralized~~)
 - b. Manager Pattern (Centralized)
4. ~~Control~~ rails
5. Context
6. Sessions
7. Tools

1. The OpenAI Agents SDK is an agentic AI framework by OpenAI.

Practical - demo

setup the environment -

create virtual environment (3.12 python version)

install packages :- openai-agents

* single-agent.py -

```
from os
from openai import OpenAI
from dotenv import load_dotenv
from agents import Agent, Runner
```

```
load_dotenv()
```

```
agent = Agent(name = "Assistant", instructions = "~")
```

```
result = Runner.run_sync(agent, "query~")
```

```
print(result.final_output)
```

↳ write query

3. (a.) multi_agent_handoff.py —

→ i — os

f — openai —

f — dotenv —

from agents import Agent

```
load_dotenv()
```

```
# one agent
```

```
history_tutor_agent = Agent(
```

```
    name = "History Tutor",
```

```
    handoff_description = "Specialist agent  
for historical questions",
```

```
    instructions = " ~ "
)
```

```
# second agent
```

```
math_tutor_agent = A — ( ~  
 ~  
 ~ )
```

```
# agent who tells which agent should handle.
```

```
allocator_agent = Agent( name = ~,  
                          instructions = ~,
```

```
    handoffs = [history_tutor_agent,  
                math_tutor_agent]
```

```
result = R — (allocator_agent, " ~ ")
```

```
print(result.final_output)
```

3(b.) multi-agent-manager.py -

imports

load the env

agents

spanish-agent = A (
 name = "Spanish agent"
 instructions = " "
)

french-agent = A (
 "French agent"
)

italian-agent = A (
)

manager-agent = A (
 name = "Manager Agent"
 in = (" "
)
)

tools = [
 spanish-agent.as-tool (
 tool-name = "translate-to-spanish"
 tool-description = "Translate user's msg to S"
) ,
 french-agent.as-tool (
) ,
 italian-agent.as-tool (
)
]

main func

async def main():

msg = "Translate 'hello' into S-, F- and J-"

orchestrator_output = await Runner.run(
manager_agent, msg)

print("\n == Final Answer == \n")

print(orchestrator_output.final_output)

if __name__ == "__main__":

asyncio.run(main())

Difference between Handoff and Manager -

↳ handoff :- when task is assigned to one agent
then output from that agent will be shown.

↳ manager :- combined outputs as an output
from all agents will be shown

4. guardrail.py -

*** Restricted questions are handled such as

"how to hack someone?" ***

→ for agents :- GuardrailFunctionOutput, Agent, Runner

for pydantic :- BaseModel

class HomeworkOutput(BaseModel):

is_homework : bool (# True / False)

reasoning : str

guardrail_agent = Agent(

name = "Guardrail check",

instructions = " ————— ", ^{whether about homework / not.}

output_type = HomeworkOutput,

)

previous agent examples

history_tutor_agent = A — (—)

math_tutor_agent = A — (—)

X allocator_agent = A — (—)

handoffs = [— , —],

input_guardrails = [InputGuardrail

(guardrail_function = homework_guardrail)

])

→ async def homework_guardrail (ctx, agent, input_data):

result = await Runner.run(guardrail_agent, input_data,

context = ctx.context)

final_output = result.final_output.as(HomeworkOutput)

return GuardrailFunctionOutput(

output_info = final_output,

tripwire_triggered = not final_output.is_homework,

) _{↳ exception for not found command.}

```

async def main():
    result = await Runner.run(AllocationAgent, "m")
    print(result.final_output)

```

```

if __name__ == "__main__":
    asyncio.run(main())

```

✱ In HomeworkOutput class, if is False then it raises exception and does not answer ✱

5. context.py -

✱ Extra information to the agent so it can provide personalized responses ✱

→ # import packages as earlier

```

import
load_dotenv()

```

```

import asy—

```

```

from dataclasses import dataclass

```

```

for agents import Agent, Runner, function_tool

```

RunContextWrapper

@dataclass

```

class UserInfo:

```

```

    name: str

```

```

    uid: int

```

```

    last_purchase: str

```

@function_tool

```

async def fetch_user_purchase(wrapper: RunContextWrapper
    [UserInfo]) → str:

```

```

    return f"The user {wrapper.content.name}

```

→ imp info
py function as tool
↑
check code to know apart from UserInfo info variable, we can extra tools, trace_id in a wrapper to the agent

recently purchased a {wrapper.content.last_purchase}."

```
async def main():
```

```
    user_info = UserInfo(name="~", uid=123,  
                          last_purchase="laptop")
```

```
    agent = Agent([UserInfo](  
        name="~"  
        instance="~"  
        tools=[fetch_user_purchase],
```

```
    result = await Runner.run(  
        starting_agent=agent,  
        input="~ query ~",  
        context=user_info,
```

```
    print("\n -- Final O/P --")
```

```
    print(result.final_output)
```

```
if __name__ == "__main__":  
    asyncio.run(main())
```

6. Session.py -

**** Session is a memory used to store previous conversations, history for specific user-id. ****

```
from agents import
```

```
from agents import Agent, Runner, SQLiteSession
```

```
in as
```

```
load_dotenv()
```

async def main():

agent = Agent(name="Assistant", in=" ")

session = SQLiteSession("conversation-123",

temporary store ← "conversation-history.db")

you can
use either
one or
both

long-term ←

result1 = await Runner.run(agent, " ", session=session)

print(result1.final_output)

result2 = ~~~~~ # Ask connected to prev query

print(~~~~~)

if ~~~~~
an()

6(b.) multi_session.py -

** If you want to work with multiple users **

→ imports

async def main():

agent = A(~~~~~)

gets stored
↑ here

session_user1 = SQLiteSession("user-123", m-session.db")

session_user2 = " ("user-456", m-session.db")

User 1

result1 = await Runner.run(agent, " ",
session=session_user1)

result2 = # same user

User 2

result3 = ~~~~~ (~~~~~, ~~~~~, session=session_user2)

if ~~~~~

7. Tools :

External functionalities which provide LLMs - which it can't do or have info.

→ Prebuilt → Custom

1. tools_demo.py - (Prebuilt)

import

from agents import Agent, Runner, WebSearchTool

```
agent = Agent(  
    name="~",  
    run = "~",  
    tools = [WebSearchTool()],  
)
```

2. function_tool_demo.py -

imports

from agents import Agent, Runner, function_tool

python functions → imp to make the func → tool

@function_tool
async def fetch_weather(city: str) → str:

city = city.lower()

if city == "san francisco":

return "~"

elif city == "delhi":

return "~"

elif city == "london":

return "~"

else:

return "f ~ {city.title()} is sunny"

```

agent = Agent(
    name = " — ",
    sim = " — ",
    tools = [fetch-weather],
)

```

```

async def main():
    result = await R_.run(agent, input = " — ")
    print("Agent response:", result.final-output)

```

```

if __name__ == '__main__':
    asyncio.run(main())

```

* Build Agentic Apps with MCP and OpenAI agents SDK

environment setup -

1. python virtual env
2. activate it

from previous 7 component file of SDK make it a folder and create virtual env there. later following are

MCP_demo/custom_mcp_server.py -

pip install openai-agents

→ import math

from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Math")

@mcp.tool()

def add(a: int, b: int) → int:

return a+b

same for multiply, divide, sq-root, factorial

```
if __name__ == "__main__":  
    mcp.run(transport = "stdio")  
        ↳ for local both  
        client & server
```

MCP Servers :

1. stdio - locally
2. HTTP over SSE - remotely, can connect via URL
3. Streamable HTTP - remotely using Streamable HTTP

transport defined in the MCP spec.

classes: MCPServerStdio, MCPServerSse, MCPServerStreamableHttp

MCP-demo/mcp-client.py -

```
# imports # load-dotenv()
```

```
from agents.mcp import MCPServerStdio
```

```
async def main():
```

```
    async with MCPServerStdio(
```

```
        params={
```

```
            "command": "python",
```

```
            "args": ["custom-mcp-server.py"]
```

```
        }
```

```
    ) as mathserver:
```

```
        agent = Agent(
```

```
            mcp_servers=[math-server]
```

```
        )
```

```
        result = await Runner.run(agent, " ~ ")
```

```
        print(result.final_output as text())
```

if

download existing server - wikipedia

pip install wikipedia-mcp

MCP_demo/multi-server-client.py -

```
# imports # load dotenv()
```

```
from agents.mcp import MCPServerStudio
```

```
math_server = MCPServerStudio(params={  
    "command": "-", "args": ["custom-mcp-server.py"]  
})
```

```
wiki_server = MCPServerStudio(params={  
    "command": "-",  
    "args": ["-m", "wikipedia-mcp"]  
})
```

```
agent = Agent(name=" ", run=" ",  
    mcp_servers=[math_server, wiki_server]  
)
```

```
async def main():
```

```
    async with math_server, wiki_server:
```

```
        runner = Runner()
```

```
        res1 = await runner.run(agent, "5?")
```

```
        print("math": res1.final_output)
```

```
        res2 =
```

```
        print("wiki": res2.final_output)
```

```
# different topics in same query
```

```
        res3 =
```

```
        print("Mixed": res3.f)
```

```
if __name__ == "__main__":
```

```
    asyncio.run(main())
```